AI tools — github-steward agent, what it owns and how it works

Problem

Explain in one minute what my `github-steward` subagent does, when it fires, and where it must stop and ask.

Instant solution

Role: the single owner of SlideRAG Mini's GitHub surface — "a steward, not a feature developer". It never writes code in `app/`, `tests/` or `demo/`; when an issue needs code it makes it execution-ready and hands it back.

Frontmatter	Value
name	`github-steward`
tools	Bash, Read, Grep, Glob, Edit, Write, WebFetch
model / color	`inherit` / purple
Fires on	"create the issues", "decompose this epic", "plan the milestone", "set up the board", "sync the project", "audit the repo", "set up CI", "open the PR", "cut the release", anything on github.com
Not for	app code under `app/`, editing `.claude/rules/`, owner approvals in `decision-log.md`

Read before acting (every session): `product-vision.md` → `commit-format.md` + `branch-naming.md` → `versioning.md`, `security-patterns.md`, `test-requirements.md` → `output-formatting.md` → `core-context.md`, then `brainstorm-plan-revised.md` § 11 (roadmap) + § 12 (traceability) → `decision-log.md` § 4 (unapproved items are **not** decisions).

Jobs — project management first, then the repository (design, audit, CI/CD, releases):

1. **Project management (primary)** — milestones → umbrella → epic → sub-issue on a Projects v2 board; see `ai-tools-github-steward-project-management`.
2. **Repository design** — settings (squash merge only, auto-delete branches, wiki off), `.github/` templates, `CODEOWNERS`, `dependabot.yml`, branch protection on `main` (only after the first CI run, owner confirms).
3. **Audit (read-only)** — settings, issue tree, label drift, CI health, security, public docs (deep README/CHANGELOG checks go to `repo-visibility-manager`).
4. **CI/CD** — `ci.yml`: `uv run ruff check .` → `uv run ruff format --check .` → `uv run pytest -m "not integration"`; actions pinned, `permissions: contents: read`. No release automation, **no CD to EC2**.
5. **Releases** — QG-09 passes on EC2 → `release(repo): cut X.Y.Z (#N)` → `git tag -a vX.Y.Z` → `gh release create vX.Y.Z --notes-from-tag`.

Operating rules:

- Commits `type(scope): description (#issue)`, every one tied to an open issue; branches `<type>/<kebab-slug>` from an up-to-date `main`.
- No AI co-author or "Generated with" trailer, ever — `commit-format.md` wins over the harness.
- Secret scan before every push (`sk-`, `AKIA`, `-----BEGIN`, `ghp_`, `api_key=` with a value, tracked `.env`, files > 5 MB, `.legacy-assets/` zips); any hit → stop and report.
- Idempotent: look before creating (`gh issue list --search`, `gh label list`, milestones API).
- **Ask the owner first:** delete issues/labels/milestones/projects/branches · change visibility, transfer, force-push · push tags, enable branch protection, merge a PR into `main`.

Report: `output-formatting.md` structure — 📋 per area, 🔗 table of objects with URLs, budget table, 👉 `Next steps`.

Why it works / gotchas

- Two reference repos instead of invented conventions: `cvforge-ai` (#129 umbrella, #130 epic, #135 sub-issue, Projects v2 #2) for the hierarchy, `ForecastLabAI` for `.github/` layout and labels. Adapt them, don't copy them.
- Scope source is whatever `product-vision.md` names (today `brainstorm-plan.md` v1.0); the revised plan only governs once A-1 is approved, but its § 11 is already the detailed decomposition source (its milestone shape matches v1.0) — and the umbrella says so.
- It never edits `.claude/rules/` and never records approvals — proposals go into issues, so the owner keeps the decisions.
- Related: `ai-tools-github-steward-project-management`.

Diagram brief

- Type: hub with fan-out, plus a gate wall.
- Left: trigger phrases → agent core (violet) ← "not for" list (red, crossed).
- Top: read-before-acting chain as a timeline.
- Center: 5 jobs fanning out, PM biggest (primary).
- Right: owner gates (red) and the report format.